

HW5

MSML606

Author: Chenhongshu Yu

UID: 116610971

Statement: The use of external resources is clearly cited. Please refer to the bold words in my submission. If no citation is found for any specific part of my submission, it means "No external sources were used; all ideas are my own or from course lecture slides".

Part I (NP-completeness)

Question 1.

TSP problem. You need to find the shortest route, visiting all cities exactly once and returning to the start. Match the following three approaches to the strongest guarantee they provide:

Algorithm Approach:

- A. Exact TSP Solver
- B. GreedyAlgorithm
- C. PureMLModel

Options:

- (1) Valid tour — visits every city exactly once and returns to start
- (2) Good tour — valid + reasonably short (guarantees proven bound, e.g., no more than 1.5x optimal solution)
- (3) Optimal tour — valid + provably shortest possible
- (4) None of the above

Provide justification.

Answer:

Exact TSP Solver matches with (3) Optimal tour.

- **Justification:** Exact TSP solvers, such as the Concorde TSP Solver, are mathematically guaranteed to find the absolute shortest valid path in the **optimal tour**. Concorde achieves this by formulating the TSP as an integer linear programming (ILP) problem and systematically searching the solution space using the cutting-plane method and branch-and-bound techniques. Because it solves the problem exactly, it provides the strongest possible guarantee, although its worst-case running time is exponential since the TSP is an NP-hard problem.
- **Citation:** Applegate, D., Bixby, R., Chvátal, V., & Cook, W. *Concorde TSP Solver*. University of Waterloo.
<https://www.math.uwaterloo.ca/tsp/concorde.html>

Question 2.

For an NP-complete problem like TSP, brute force is $O(n!)$. A trained ML model can produce a solution in milliseconds. Does that mean ML solved TSP efficiently? What's the difference between a good solution and an optimal solution?

Answer:

No, that a trained ML model can produce a solution in milliseconds does not mean it has "solved" the Traveling Salesman Problem (TSP) efficiently. In theoretical computer science, to "solve" an NP-complete problem efficiently specifically implies finding the absolute, guaranteed *optimal* solution in polynomial time ($\mathcal{O}(n^k)$). When an ML model outputs a route in milliseconds, it is simply performing forward-pass inference. It acts as a fast heuristic that leverages learned patterns to output a highly probable guess. It is bypassing the exhaustive search required to find the true mathematical minimum, intentionally trading exactness (optimality) for execution speed. Therefore, it has not broken the theoretical barrier of NP-completeness, which it is just quickly calculating an approximation.

Assumption: I assume that the phrase "solved efficiently" refers to the formal computational complexity definition (finding the exact answer in polynomial time) and that the ML model is acting purely as a predictive heuristic without pairing it with an exact combinatorial solver.

Citation: Applegate, D., Bixby, R., Chvátal, V., & Cook, W. *Concorde TSP Solver*. University of Waterloo.
<https://www.math.uwaterloo.ca/tsp/concorde.html>

Question 3.

If ML can approximate an NP-hard problem well in practice, does that mean the problem is no longer hard? What does NP-hardness actually say about ML approximations?

Answer:

No, an ML model approximating an NP-hard problem well in practice does not change the problem's theoretical idea of what the NP-hard question is. NP-hardness is a rigorous worst-case computational complexity guarantee indicating that no algorithm (including an ML neural network) can effectively find an exact, optimal solution for *all* possible instances of the problem in polynomial time. While an ML model might use learned heuristics to quickly find "good enough" practical approximations for standard data distribution patterns (the average case), NP-hardness dictates that adversarial or highly complex edge-case inputs will always exist that break the ML approximation and force it to compute sub-optimal or invalid results, proving the fundamental mathematical hardness remains intact.

Assumption: I assume "in practice" refers to performing well on typical average-case data distributions rather than theoretical worst-case edge instances.

Citation: GeeksforGeeks. (2023). "Difference between NP hard and NP complete problem." Available at: <https://www.geeksforgeeks.org/dsa/difference-between-np-hard-and-np-complete-problem/> (Explains worst-case complexity bounds in contrast to practical heuristics).

Question 4.

How would you evaluate whether an ML solution to an NP-hard problem is good if you can't efficiently compute the optimal solution to compare against?

Answer:

To evaluate an ML solution for an NP-hard problem without known optimal solutions, people typically compare the model's output to mathematical lower bounds to calculate an expected optimality gap, or they benchmark the solution against robust traditional heuristic algorithms to measure relative speed and accuracy. Furthermore, these ML models are rigorously validated on established benchmark datasets like TSPLIB, a collection of hard problem instances where the true optimal solutions have already been computed by supercomputers.

Assumption: I assume the context involves combinatorial optimization where the problem's objective function can be mathematically relaxed to find a theoretically guaranteed minimum lower-bound for comparison.

Citation: Bello, I., Pham, H., Le, Q. V., Norouzi, M., & Bengio, S. (2016). "Neural Combinatorial Optimization with Reinforcement Learning." arXiv paper. Available at: <https://arxiv.org/abs/1611.09940> (This paper details the standard methods of using benchmark datasets like TSPLIB and routing approximation baselines to evaluate ML combinatorial models).

Part II (Order statistics)

Problem 1.

Use QuickSelect algorithm to trace:

- for a random input
- and for a sorted input

Requirements: use at least 6-8 elements so your traces are meaningfully illustrative

Answers to Problem 1: QuickSelect Tracing

QuickSelect Trace (Sorted Input)

Use input: [10, 20, 30, 40, 50, 60, 70] (7 elements)

Goal: Find the 4th smallest element (target is 40, target sorted index is 3).

Assumption: Pivot is always the last element of the active subarray.

1. Active array: [10, 20, 30, 40, 50, 60, 70], target index 3, pivot is 70
 - Partition places 70 at relative index 6 -> [10, 20, 30, 40, 50, 60, 70]
 - Target index 3 < 6 -> recurse left
2. Active array: [10, 20, 30, 40, 50, 60], target index 3, pivot is 60
 - Partition places 60 at relative index 5 -> [10, 20, 30, 40, 50, 60]
 - Target index 3 < 5 -> recurse left
3. Active array: [10, 20, 30, 40, 50], target index 3, pivot is 50
 - Partition places 50 at relative index 4 -> [10, 20, 30, 40, 50]

- Target index $3 < 4 \rightarrow$ recurse left
4. Active array: [10, 20, 30, 40] , target index 3 , pivot is 40
- Partition places 40 at relative index 3 $\rightarrow$ [10, 20, 30, 40]
 - Target index $3 == 3 \rightarrow$ **return 40.**

QuickSelect Trace (Random Input)

Use input: [35, 12, 89, 42, 21, 64, 7] (7 elements)

Goal: Find the 4th smallest element (target is 35 , target sorted index is 3).

Assumption: Pivot is always the last element of the active subarray.

1. Active array: [35, 12, 89, 42, 21, 64, 7] , target index 3 , pivot is 7
 - Partition $\rightarrow$ [7, 35, 12, 89, 42, 21, 64]
 - Pivot 7 is at relative index 0
 - Target index $3 > 0 \rightarrow$ recurse right (discard 1 left element)
2. Active array: [35, 12, 89, 42, 21, 64] , new relative target index $= 3 - 1 = 2$, pivot is 64
 - Partition $\rightarrow$ [35, 12, 42, 21, 64, 89]
 - Pivot 64 is at relative index 4
 - Target index $2 < 4 \rightarrow$ recurse left
3. Active array: [35, 12, 42, 21] , target index remains 2 , pivot is 21
 - Partition $\rightarrow$ [12, 21, 42, 35]
 - Pivot 21 is at relative index 1
 - Target index $2 > 1 \rightarrow$ recurse right (discard 2 left elements)
4. Active array: [42, 35] , new relative target index $= 2 - 2 = 0$, pivot is 35
 - Partition $\rightarrow$ [35, 42]
 - Pivot 35 is at relative index 0
 - Target index $0 == 0 \rightarrow$ **return 35.**

Final output: 35

Problem 2.

Suppose we are using the QuickSelect algorithm on a worst case input sequence. Pivot always lands at the smallest or largest element.

- Write the recurrence relation $T(n)$
- Draw recursion trees
- Solve the recurrence

Answers to Problem 2: QuickSelect Worst-Case Recurrence

Recurrence Relation

In the worst-case scenario, the pivot always lands at the smallest or largest element, discarding only exactly one element (the pivot). We must then recurse on the remaining $n - 1$ elements. Partitioning still takes linear time ($\mathcal{O}(n)$ or cn).

Recurrence Relation: $T(n) = T(n - 1) + cn$

Recursion Tree

Because only one recursion is made per step, there is exactly **1 node** at each level.

- **Level 0:** Size n
 - Work done: cn
 - Nodes: 1
- **Level 1:** Size $n - 1$
 - Work done: $c(n - 1)$
 - Nodes: 1
- **Level 2:** Size $n - 2$
 - Work done: $c(n - 2)$
 - Nodes: 1
- **Level 3:** Size $n - 3$
 - Work done: $c(n - 3)$
 - Nodes: 1
- **...and so on down to base case size 1.**

Solving the Recurrence

To find the total worst-case time complexity, we sum up the work have done across all levels of the unbalanced tree:

$$T(n) = \sum_{i=1}^n ci = c(n) + c(n-1) + c(n-2) + \cdots + c(1)$$

$$T(n) = c \sum_{i=1}^n i = c \left(\frac{n(n+1)}{2} \right)$$

$$T(n) = c \left(\frac{n^2}{2} + \frac{n}{2} \right)$$

Dropping the constants and lower-order terms yields:

Solution: $\mathcal{O}(n^2)$

Problem 3.

Suppose we are using the Median-of-medians algorithm.

- Write the recurrence relation $T(n)$
- Draw recursion trees
- Solve the recurrence

Answers to Problem 3: Median-of-medians Recurrence

Recurrence Relation

In the Median-of-medians algorithm (with groups of 5), we divide the array of size n into groups of 5 to find their medians, and then recursively call the algorithm to find the median of those $\frac{n}{5}$ medians. Once we have the true "median of medians" (the pivot), we partition the original array. In the worst-case scenario, partitioning around this pivot guarantees that at least $\frac{3n}{10}$ elements are discarded, leaving a maximum of $\frac{7n}{10}$ elements for the main recursive call. The initial dividing, finding medians, and partitioning work takes linear time cn .

Recurrence Relation: $T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + cn$

Recursion Tree

Because there are two recursive calls of different fraction sizes, the tree is unbalanced, branching into 2 at every step.

- **Level 0:**

- Node Sizes: n
- Number of Nodes: 1
- Work Done: cn

- **Level 1:**

- Node Sizes: $\frac{n}{5}$ and $\frac{7n}{10}$
- Number of Nodes: 2
- Work Done: $c \left(\frac{n}{5} \right) + c \left(\frac{7n}{10} \right) = c \left(\frac{2n}{10} + \frac{7n}{10} \right) = \frac{9}{10}cn$

- **Level 2:**

- Node Sizes: $\frac{n}{25}$, $\frac{7n}{50}$, $\frac{7n}{50}$, and $\frac{49n}{100}$
- Number of Nodes: 4
- Work Done: $c \left(\frac{n}{25} + \frac{7n}{50} + \frac{7n}{50} + \frac{49n}{100} \right) = c \left(\frac{4n+14n+14n+49n}{100} \right) = \frac{81}{100}cn = \left(\frac{9}{10} \right)^2 cn$

Solving the Recurrence

To find the worst-case total time, we sum the work done across all levels of the tree. Since the total work shrinks by a factor of $\frac{9}{10}$ at each level, this forms a converging geometric series.

$$T(n) \leq \sum_{i=0}^{\infty} cn \left(\frac{9}{10} \right)^i$$

Using the infinite geometric series formula $\sum_{i=0}^{\infty} r^i = \frac{1}{1-r}$ (where $r = 0.9$):

$$T(n) \leq cn \left(\frac{1}{1-0.9} \right) = cn \left(\frac{1}{0.1} \right) = 10cn$$

Since $10c$ is just a constant multiplier, we drop it to find the final time complexity. **Solution:** $\mathcal{O}(n)$

Problem 4.

Suppose we develop a pivot-selection algorithm (like Median-of-medians) that guarantees the pivot falls within the 10th to 90th percentile. That means that at least 10% of the elements are discarded in every recursive step. Assume that Partitioning still takes $\mathcal{O}(n)$.

- Write the recurrence relation $T(n)$ representing the worst-case running time for this algorithm.
- Draw recursion trees
- Solve the recurrence

Answers to Problem 4: Custom Algorithm Recurrence

Worst-Case Recurrence Relation

Because this pivot-selection algorithm guarantees the pivot falls within the 10th and 90th percentiles, the worst-case split will leave 90% ($\frac{9n}{10}$) of the elements on one side of the partition and 10% on the other. Just like in QuickSelect, we only recurse on *one* side to find the target element (unlike Quicksort which recurses on both). The worst-case is that we must always recurse into the larger 90% remaining portion. The partitioning process takes $\mathcal{O}(n)$ time (or cn).

Recurrence Relation: $T(n) \leq T\left(\frac{9n}{10}\right) + cn$

Recursion Tree

Because this is a selection algorithm that only follows exactly one path in the search space, the recursion tree does not branch out, which it only descends in a single straight line. Let's trace the worst-case path down the tree where we always pick the $0.9n$ branch.

- **Level 0:**
 - Node Sizes: n
 - Number of Nodes: 1
 - Work Done: cn
- **Level 1:**
 - Node Sizes: $\frac{9n}{10}$
 - Number of Nodes: 1
 - Work Done: $c\left(\frac{9n}{10}\right) = \frac{9}{10}cn$
- **Level 2:**
 - Node Sizes: $\frac{81n}{100}$ (or $\left(\frac{9}{10}\right)^2 n$)
 - Number of Nodes: 1
 - Work Done: $c\left(\frac{81n}{100}\right) = \left(\frac{9}{10}\right)^2 cn$

Solving the Recurrence

To find the worst-case time complexity, we sum up the work have done across the depth of the tree up to infinity (since it's bounded). The series shrinks at an exact factor of $9/10$, forming a simple convergence:

$$T(n) \leq \sum_{i=0}^{\infty} cn \left(\frac{9}{10} \right)^i$$

Using the infinite geometric series formula $\sum_{i=0}^{\infty} r^i = \frac{1}{1-r}$ (where the ratio $r = 0.9$):

$$T(n) \leq cn \left(\frac{1}{1 - 0.9} \right)$$

$$T(n) \leq cn \left(\frac{1}{0.1} \right) = 10cn$$

Since $10c$ is just a constant multiplier reflecting the linear work overhead, we drop it.

Solution: $\mathcal{O}(n)$